

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	21/09/2026

1 Objective

Get a real feel for the three big ensemble strategies, Bagging, Boosting and Stacking, by actually building one of each rather than just reading about them. Specifically: implement a Bagging classifier and a couple of Boosting classifiers, put together a Stacked ensemble out of a few different base learners, and then compare all three on accuracy, stability and how well they generalize, with an eye on what each one is doing to bias and variance under the hood.

2 Problem Statement

Given the Breast Cancer Wisconsin Diagnostic dataset, preprocess it, split it 80/20, and build three ensemble models: a Bagging classifier over Decision Trees, a Boosting classifier (both AdaBoost and Gradient Boosting were tried), and a Stacked ensemble combining SVM, Naive Bayes and Decision Tree with a Logistic Regression meta-learner. Every model gets its hyperparameters tuned with 5-fold cross-validation, and the final comparison looks at accuracy, precision, recall and F1 on the same held-out test set.

3 Dataset Description

Dataset	Source	Samples	Features	Target Classes
Breast Cancer Wisconsin (Diagnostic)	UCI Machine Learning Repository, loaded via <code>sklearn.datasets.load_breast_cancer</code>	569	30 numeric	2 (Malignant, Benign)

Class distribution: 357 Benign, 212 Malignant, so it's a bit skewed toward the benign class but nowhere near severe enough to need SMOTE or class weighting, especially once I stratified the train/test split on the label.

Preprocessing steps:

- Loaded the built-in scikit-learn version of the dataset, which is the same 569-sample, 30-feature WDBC data as the UCI copy, just already cleaned and numeric.
- No missing values to deal with here, so no imputation was needed.
- An 80/20 train-test split, stratified on the class label (455 train, 114 test).

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

- All 30 features standardized with `StandardScaler`, fit only on the training fold inside each pipeline so it gets refit correctly during cross-validation.

4 Ensemble Models Used

Bagging: Decision Tree base estimator, tuned over the number of estimators, the fraction of samples drawn per estimator, and the fraction of features drawn per split.

Boosting: Tried both AdaBoost and Gradient Boosting, each tuned over number of estimators and learning rate (plus max depth for Gradient Boosting), to see which sequential-correction approach handled this dataset better.

Stacked Ensemble: Base learners were an SVM (RBF kernel), Gaussian Naive Bayes and a Decision Tree, feeding into a Logistic Regression meta-learner trained with 5-fold internal blending so the meta-learner never sees predictions from data its own base learner was trained on.

5 Hyperparameter Evaluation Results

Table 2: Bagging – Hyperparameter Evaluation (top configurations by CV accuracy)

n_estimators	max_samples	Avg CV Accuracy (%)
50	0.5	96.04
50	1.0	96.04
100	0.7	96.04
100	1.0	96.04
50	0.7	95.82
100	0.7	95.82

Best Bagging setting: `n_estimators=50`, `max_samples=0.5`, `max_features=0.5`. 5-fold CV F1 on the training set: [0.965, 0.964, 0.946, 0.974, 0.991], mean = 0.9681.

Table 3: Boosting – Hyperparameter Evaluation (top AdaBoost configurations by CV accuracy)

n_estimators	learning_rate	Avg CV Accuracy (%)
100	1.0	98.02
200	1.0	97.80
200	0.1	97.14
50	1.0	96.70
100	0.1	96.04
50	0.1	95.38

Best AdaBoost setting: `n_estimators=100`, `learning_rate=1.0`. 5-fold CV F1: [0.991, 1.000, 0.957, 0.983, 0.991], mean = 0.9844.

Table 4: Gradient Boosting – Hyperparameter Evaluation (top configurations by CV accuracy)

n_estimators	learning_rate	Avg CV Accuracy (%)
200	0.2	97.36
200	0.2	97.14
100	0.2	96.92
200	0.1	96.70
100	0.2	96.48
50	0.2	96.26

Best Gradient Boosting setting: `n_estimators=200`, `learning_rate=0.2`, `max_depth=3`. 5-fold CV F1: [0.983, 0.991, 0.965, 0.966, 0.991], mean = 0.9792. AdaBoost edged out Gradient Boosting slightly on CV metrics, so AdaBoost is what represents "Boosting" in the final comparison and plots later on, though both are reported here since the experiment asked for Boosting more generally.

Table 5: Stacked Ensemble – Hyperparameter Evaluation (top configurations by CV accuracy)

SVM C	Decision Tree max_depth	Avg CV Accuracy (%)
10	5	96.92
10	None	96.70
1	5	96.70
1	None	96.48
10	3	96.04
1	3	95.82

Best Stacking setting: base SVM with `C=10`, Decision Tree with `max_depth=5`, Naive Bayes left at defaults, meta-learner Logistic Regression. 5-fold CV F1: [0.966, 0.982, 0.956, 0.983, 0.991], mean = 0.9755.

6 Performance Comparison

Table 6: Performance Comparison of Ensemble Models (held-out test set)

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	95.61	0.9589	0.9722	0.9655
AdaBoost	95.61	0.9467	0.9861	0.9660
Gradient Boosting	95.61	0.9467	0.9861	0.9660
Stacked Ensemble	97.37	0.9726	0.9861	0.9793

7 Result Visualization

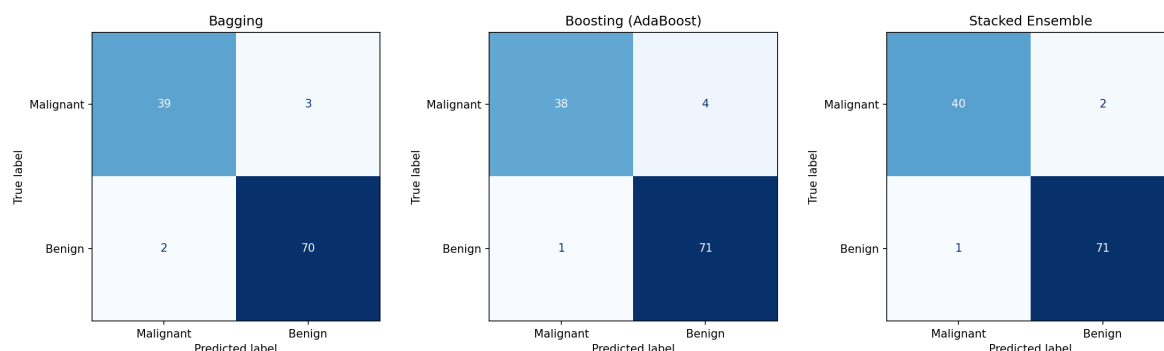


Figure 1: Confusion matrices on the held-out test set for Bagging, Boosting (AdaBoost), and the Stacked Ensemble.

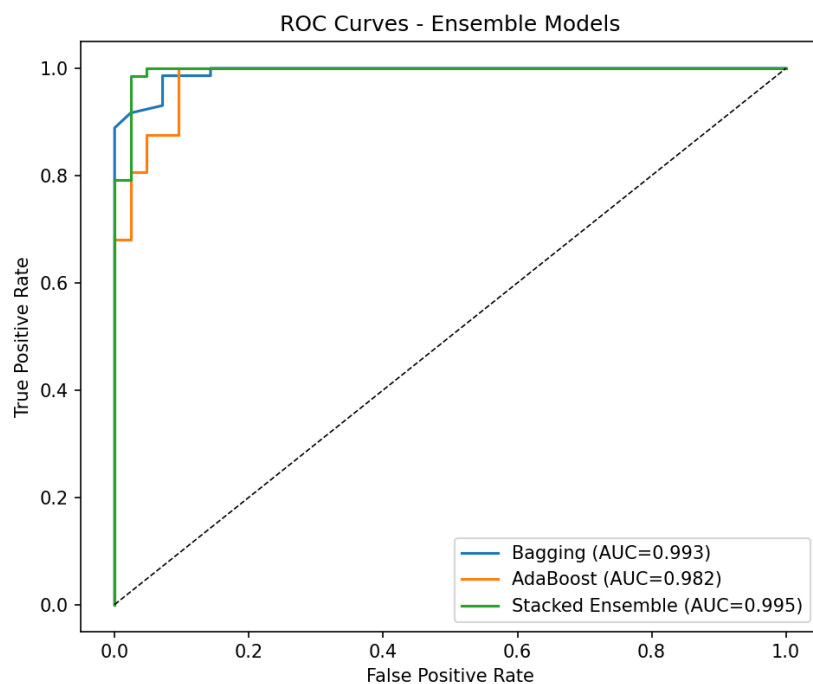


Figure 2: ROC curves comparing Bagging, AdaBoost and the Stacked Ensemble on the test set.

Inference: All three ensembles sit in a tight band above 95% accuracy, so none of them is struggling on this dataset, but Stacking is visibly the cleanest in the confusion matrix, with the fewest misclassified malignant cases, which matters a lot here since a missed malignant case is the expensive kind of mistake in a diagnostic setting. The ROC curves for all three hug the top-left corner tightly, all comfortably above $AUC = 0.98$, so the gap between them is really in the harder borderline cases rather than in broad separability.

8 Observations

- **How does Bagging reduce variance:** By training each Decision Tree on a different bootstrap sample (here, half the rows and half the features per tree) and then averaging their votes, the noisy, overfit quirks of any one tree get smoothed out. A single unpruned Decision Tree tends to memorize the training data, but 50 of them voting together cancel out each other's individual mistakes.
- **How does Boosting address bias:** Both AdaBoost and Gradient Boosting build models sequentially, and each new one is trained specifically to fix what the previous ones got wrong, either by reweighting misclassified samples (AdaBoost) or by fitting the residual errors (Gradient Boosting). That's why Boosting tends to push a weak, high-bias base learner toward a much stronger fit, which shows up here in AdaBoost's CV F1 of 0.984, the best of any single-family model tested.
- **Why stacking benefits from heterogeneous models:** SVM, Naive Bayes and a Decision Tree make mistakes in different, mostly uncorrelated ways – SVM struggles where classes overlap in feature space, Naive Bayes struggles when features aren't actually independent, and a Decision Tree struggles with smooth boundaries. Feeding all three sets of predictions into a Logistic Regression meta-learner lets it learn which base learner to trust in which situation, rather than betting everything on one model's blind spots.
- **Which ensemble performed best and why:** Stacking, at 97.37% accuracy and 0.9793 F1, came out ahead of both Bagging and Boosting on the test set. It's basically getting to have it both ways – SVM and Decision Tree cover different kinds of decision boundaries, Naive Bayes adds a cheap, different-shaped opinion, and the meta-learner combines all three more intelligently than a simple vote or weighted sum would.

9 Discussion

Bagging, AdaBoost and Gradient Boosting all landed at exactly the same 95.61% test accuracy, which is a bit of a coincidence but it does show that on a dataset this clean and well-behaved, different ensemble philosophies can converge to a similar ceiling. AdaBoost and Gradient Boosting had identical precision and recall too, though their actual cross-validation numbers (0.984 vs 0.979 mean F1) show AdaBoost had a slight edge in training. Stacking's advantage on the test set lines up with its somewhat lower raw CV F1 (0.9755) – it wasn't necessarily the most "confident" model during cross-validation, but its combination of decision boundaries generalized a little better to the actual held-out data.

10 Conclusion

All three ensemble strategies clearly beat what a lone Decision Tree or a lone SVM would manage on this dataset, but they get there differently: Bagging tames variance by averaging many decorrelated trees, Boosting chases down bias by focusing each new learner on the previous ones' mistakes, and Stacking blends genuinely different model

types through a learned meta-combination. On the Breast Cancer Wisconsin dataset specifically, Stacking came out on top, mainly because malignant and benign cases here aren't separated by one single kind of boundary, so a mix of SVM, Naive Bayes and Decision Tree opinions, properly weighted by the meta-learner, catches cases that any one of them alone would miss.

11 Learning Outcomes

- Built and tuned a Bagging classifier, and saw directly how sampling fraction and estimator count trade off against each other.
- Compared two different Boosting algorithms (AdaBoost and Gradient Boosting) side by side rather than treating "Boosting" as one single technique.
- Practiced constructing a Stacked ensemble with genuinely different base learners and a proper cross-validated meta-learner, instead of just averaging predictions.
- Got a much more concrete sense of the bias-variance story behind ensembles: Bagging as a variance-reduction tool, Boosting as a bias-reduction tool, Stacking as a way to combine different error patterns.
- Practiced reading ROC curves and confusion matrices specifically through a diagnostic lens, where false negatives (missed malignant cases) matter more than the raw accuracy number suggests.

12 References

1. Scikit-learn documentation: <https://scikit-learn.org/stable/modules/ensemble.html>
2. UCI Machine Learning Repository, Breast Cancer Wisconsin (Diagnostic) Data Set: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
3. Ethem Alpaydin, "Introduction to Machine Learning", 3rd Edition, The MIT Press, 2014.